Differentiable Particle Filtering for Bayesian SSM Inference

HMC + LEDH-invertible + OT resampling

Haowu Duan

May 7, 2026

Roadmap

Big picture: HMC + OT + LEDH-invertible

Sequential Monte Carlo and LEDH-invertible

OT resampling, Sinkhorn, and the neural operator

HMC + LEDH + OT pipeline

Big picture: HMC + OT + LEDH-invertible

The problem we are solving

Bayesian inference for state-space models.

$$x_t = f_\theta(x_{t-1},\, \eta_t)$$
$$z_t = h_\theta(x_t,\, \varepsilon_t)$$

Given $z_{1:T}$, recover the posterior $p(\theta \mid z_{1:T}) \propto p(z_{1:T} \mid \theta)\, p(\theta)$.

The hard piece: $p(z_{1:T} \mid \theta)$ has no closed form for nonlinear / non-Gaussian models (stochastic volatility, range-bearing tracking).

Our approach

- LEDH-invertible particle flow particle filter for the proposal.
- Optimal transport (Sinkhorn) resampling to make $p(z_{1:T} \mid \theta)$ differentiable in θ.
- Hamiltonian Monte Carlo to explore the posterior, validated on 1D linear Gaussian, 1D stochastic volatility, and range-bearing.

Sequential Monte Carlo and LEDH-invertible

Filtering as integration

Filtering does not need the whole posterior $p(x_t \mid z_{1:t})$ — it needs **moments**:

$$\mathbb{E}[f(x_t) \mid z_{1:t}] \;=\; \int f(x_t)\, p(x_t \mid z_{1:t})\, dx_t,$$

in particular the mean m_t and covariance P_t.

The recursion

$$p(x_t \mid z_{1:t}) \;\propto\; p(z_t \mid x_t) \int p(x_t \mid x_{t-1})\, p(x_{t-1} \mid z_{1:t-1})\, dx_{t-1}$$

is a **high-dimensional path integral**. Closed-form only in the linear-Gaussian case (Kalman). Otherwise intractable.

Particle methods are an integration scheme. Represent the filtering distribution by weighted samples $\{(x_t^i, w_t^i)\}_{i=1}^N$, $\sum_i w_t^i = 1$. Any moment becomes a weighted sum:

$$\mathbb{E}[f(x_t) \mid z_{1:t}] \;\approx\; \sum_i w_t^i\, f(x_t^i).$$

Sequential Monte Carlo — the recipe

At step t, given $\{(x_{t-1}^i, w_{t-1}^i)\}$ approximating $p(x_{t-1} \mid z_{1:t-1})$ with $\sum_i w_{t-1}^i = 1$:

1. **Propagate** from any proposal: $x_t^i \sim q(\cdot \mid x_{t-1}^i, z_t)$.
2. **Reweight** (unnormalised):

$$\tilde{w}_t^i \;=\; w_{t-1}^i \cdot \frac{p(z_t \mid x_t^i)\, p(x_t^i \mid x_{t-1}^i)}{q(x_t^i \mid x_{t-1}^i, z_t)}.$$

3. **Normalise:** $w_t^i \;=\; \tilde{w}_t^i \,/\, \sum_j \tilde{w}_t^j$.
4. **Resample** when $\mathrm{ESS} < 0.5\,N$.

Marginal likelihood — the integral $\int p(z_{1:T}, x_{1:T} \mid \theta)\, dx_{1:T}$ — estimated by

$$\log p(z_{1:T} \mid \theta) \;=\; \sum_{t=1}^{T} \log\left(\sum_i \tilde{w}_t^i \right).$$

Unbiased in p, but **noisy and non-smooth in** θ when resampling is multinomial.

Why the standard PF gradient hurts HMC

Two problems for $\nabla_\theta \log p$:

- **Resampling kills the gradient.** Multinomial / systematic resampling chooses indices — discrete, non-differentiable. The score-function trick gives high-variance gradients.
- **Bootstrap proposal is wasteful.** $q = p(\cdot \mid x_{t-1})$ ignores z_t; weights collapse, ESS crashes, variance of ∇_θ explodes.

Two problems, two fixes:

Better proposal $\rightarrow$	LEDH flow (this section)
Differentiable resample $\rightarrow$	OT (next section)

LEDH flow: the idea

Bayesian update at step k:

$$p(x_k \mid z_{1:k}) \;\propto\; \underbrace{p(x_k \mid z_{1:k-1})}_{g(x)} \cdot \underbrace{p(z_k \mid x_k)}_{h(x)}.$$

BPF handles this by reweighting g by h — the source of weight degeneracy. Daum–Huang takes the other route: **move the particles** via a deterministic flow that transports samples of g into samples of gh.

Pseudo-time $\lambda \in [0, 1]$, **log-homotopy**:

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x), \qquad p(\cdot, 0) = g, \quad p(\cdot, 1) \propto gh.$$

Particles obey $dx/d\lambda = f(x, \lambda)$. The induced density evolution is the **(zero-diffusion) Fokker–Planck / continuity equation**:

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf).$$

LEDH flow: the implementation

Many flows f satisfy the continuity equation. Pick a tractable one: **affine ansatz** with a Gaussian assumption for $p(x, \lambda)$.

Affine ansatz. $f(x, \lambda) = A(\lambda)\, x + b(\lambda)$.

Plugging into the Fokker–Planck constraint with the Gaussian assumption $p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda)$ and matching the precision evolution $P_\lambda^{-1} = P^{-1} + \lambda H^\top R^{-1} H$ gives a closed form:

$$A(\lambda) = -\tfrac{1}{2} P\, H^\top (\lambda H P H^\top + R)^{-1} H, \quad b(\lambda) = (I + 2\lambda A)\big[(I + \lambda A) P H^\top R^{-1} z + A \bar{\eta}_0\big].$$

$H = \partial h_\theta / \partial x$ is the observation-function Jacobian.

LEDH (vs. EDH). Re-evaluate $H^{(i)} = \partial h_\theta / \partial x \big|_{x^{(i)}(\lambda)}$ at *each particle's own current position* at every λ step. Each particle gets its own $A^{(i)}(\lambda)$, $b^{(i)}(\lambda)$.

Why invertibility matters here

Invertible variant. Track the Jacobian along the trajectory:

$$J_t^i = \prod_k (I + \Delta\lambda\, A^{(i)}_{\lambda_k}).$$

Weight update uses $\log|\det J_t^i|$ instead of a free proposal density.

- Free proposal q: weight ratio needs $q(x_t \mid x_{t-1}, z_t)$, which we do not have for a flow.
- Invertible flow: $\log|\det J_t^i|$ enters the weight update via change-of-variables, accounting for the discretisation error the flow introduces in the measure.
- J_t^i is differentiable in θ. So the entire propagation step contributes a clean ∇_θ.

Practical note. TF's standard log-determinant routine swaps matrix rows around as it computes; JIT compilation on CPU does not know how to handle that pattern, so it errors. We compute $\log|\det J|$ from a QR factorisation instead — no row-swapping, JIT-friendly on every backend.

LEDH-PF in this codebase

- Filter takes θ as an explicit tensor input, never as a captured constant. This avoids retracing on every HMC proposal.
- Inner loop over $T \times n_\lambda$ is a graph-mode while-loop, so T becoming dynamic does not retrace.
- Diagnostics use graph-mode tensor arrays, not Python lists. Logging does not break the trace.
- Test coverage: dedicated JIT-compile and value-and-gradient suites.

Status: forward and gradient verified. JIT speedup $\sim 1.3\times$ forward at $T{=}20$. Throughput is dominated by $T \times n_\lambda \times N$, not framework overhead, except in 1D/2D models.

OT resampling, Sinkhorn, and the neural operator

Two problems with classical resampling

Resampling should change the particle representation *without* changing the distribution it represents. Classical schemes have two failure modes.

Problem 1 — sampling noise. Multinomial / systematic resampling draws indices from $\mathbf{w}$. The selection itself injects extra Monte-Carlo noise on top of the filter's own variance. Sample impoverishment kills covariance estimates.

Problem 2 — non-differentiability. Picking discrete indices is not differentiable in any input. Standard linear-program OT solvers fix Problem 1 (deterministic) but not Problem 2 (LP plan has no derivatives w.r.t. inputs).

Entropy-regularised OT (next slides) addresses both: deterministic transport map, differentiable in $(\mathbf{x}, \mathbf{w})$.

Reich's framing of resampling

Reich (2013), A nonparametric ensemble transform method for Bayesian inference (SIAM J. Sci. Comput. 35(4)). Treat resampling as a *transport* problem. Pre-resample weighted cloud $\{(x_i, w_i)\}$, target equal-weight cloud $\{(\hat{x}_i, 1/N)\}$.

Solve the discrete OT with marginals

$$\mathbf{a} = \mathbf{w}, \qquad \mathbf{b} = \tfrac{1}{N}\mathbf{1}_N, \qquad C_{ij} = \tfrac{1}{2}\|x_i - x_j\|^2.$$

Optimal coupling $\mathbf{P}^*$ has row sums $\mathbf{w}$ (source), column sums $\frac{1}{N}\mathbf{1}_N$ (target).

Resampled particle = column-weighted barycentre:

$$\hat{x}_i \;=\; N \sum_j P^*_{ji}\, x_j \;=\; \sum_j T_{ij}\, x_j, \qquad T_{ij} = N\, P^*_{ji}.$$

Each new particle is a convex combination of the old ones. **No discrete selection** — every input particle contributes to every output particle through T_{ij}.

This is the gateway to differentiability: T depends smoothly on $(\mathbf{x}, \mathbf{w})$ via the Sinkhorn solve.

Reich's consistency theorem

Why is the output a faithful sample of the underlying target q?

Sinkhorn itself is agnostic — give it any $(\mathbf{a}, \mathbf{b}, \mathbf{C})$ and it returns the OT plan. The reason the resampled cloud is asymptotically a realisation of q is a consistency result.

Theorem (Reich, A nonparametric ensemble transform method for Bayesian inference, 2013). Particles $x^{(1)}, \dots, x^{(N)}$ i.i.d. from proposal q_{prop}, importance weights $w_i \propto q(x^{(i)})/q_{\text{prop}}(x^{(i)})$. Let $\mathbf{P}^*$ be the unregularised OT plan with marginals $(\mathbf{w}, \frac{1}{N}\mathbf{1})$ and L^2 cost. Define $\hat{x}^{(i)} = N \sum_j P^*_{ji} x^{(j)}$. Then for every continuous bounded f,

$$\tfrac{1}{N} \sum_i f(\hat{x}^{(i)}) \xrightarrow{N\to\infty} \int f \, dq \quad \text{a.s.}$$

Equivalently, $\frac{1}{N} \sum_i \delta_{\hat{x}^{(i)}} \rightharpoonup q$ weakly.

Vanilla Sinkhorn — where the iteration comes from

Setup. Two probability vectors $\mathbf{a} \in \Sigma_M$, $\mathbf{b} \in \Sigma_N$. Coupling matrices

$$\mathcal{U}(\mathbf{a}, \mathbf{b}) = \{\, \mathbf{P} \in \mathbb{R}_+^{M \times N} \,:\, \mathbf{P}\mathbf{1}_N = \mathbf{a},\ \mathbf{P}^\top \mathbf{1}_M = \mathbf{b} \,\}.$$

Entropic-regularised OT (Cuturi 2013):

$$\min_{\mathbf{P} \in \mathcal{U}(\mathbf{a},\mathbf{b})} \langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H}(\mathbf{P}), \qquad \mathbf{H}(\mathbf{P}) = -\sum_{ij} \mathbf{P}_{ij}(\log \mathbf{P}_{ij} - 1).$$

KKT. Lagrangian $\mathcal{L} = \langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H} - \langle \mathbf{f}, \mathbf{P}\mathbf{1}_N - \mathbf{a} \rangle - \langle \mathbf{g}, \mathbf{P}^\top \mathbf{1}_M - \mathbf{b} \rangle$. Setting $\partial \mathcal{L} / \partial \mathbf{P}_{ij} = 0$:

$$\mathbf{P}_{ij} = \exp\!\left(\tfrac{\mathbf{f}_i}{\varepsilon}\right) \exp\!\left(-\tfrac{\mathbf{C}_{ij}}{\varepsilon}\right) \exp\!\left(\tfrac{\mathbf{g}_j}{\varepsilon}\right) \;=\; \mathbf{u}_i \, \mathbf{K}_{ij} \, \mathbf{v}_j.$$

Derivation: Xie (2025), *Deriving the Gradients of Some Popular OT Algorithms* (arXiv:2504.08722), Prop. 3.

Vanilla Sinkhorn — the algorithm

In index notation, the marginal constraints $\sum_j P_{ij} = a_i$ and $\sum_i P_{ij} = b_j$ on $P_{ij} = u_i K_{ij} v_j$ give

$$u_i\, K_{ij}\, v_j \;=\; a_i \quad (\text{sum on } j), \qquad v_j\, K_{ij}\, u_i \;=\; b_j \quad (\text{sum on } i),$$

which solve to

$$u_i \;=\; \frac{a_i}{K_{ij}\, v_j}, \qquad v_j \;=\; \frac{b_j}{K_{ij}\, u_i}.$$

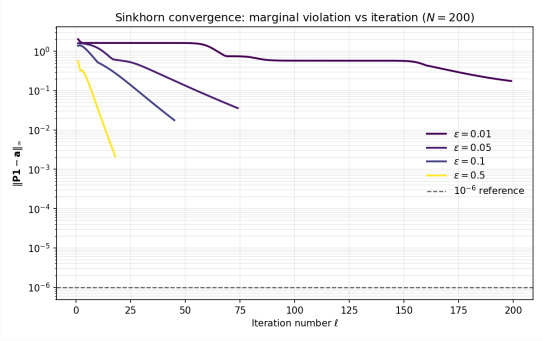
Algorithm (Vanilla Sinkhorn, Xie 2025, Alg. 3.1)
Input: $a_i, b_j, C_{ij}, \varepsilon > 0$. **Init:** $u_i = 1$, $v_j = 1$. $K_{ij} = \exp(-C_{ij}/\varepsilon)$.
Repeat until convergence: $u_i \leftarrow a_i/(K_{ij}\, v_j)$; $v_j \leftarrow b_j/(K_{ij}\, u_i)$.
Output: $P_{ij} = u_i\, K_{ij}\, v_j$.

Stopping criterion: $|u_i^{(L)} K_{ij} v_j^{(L)} - a_i| \le \rho$ and $|v_j^{(L)} K_{ij} u_i^{(L)} - b_j| \le \rho$ for every free index.

Convergence of vanilla Sinkhorn

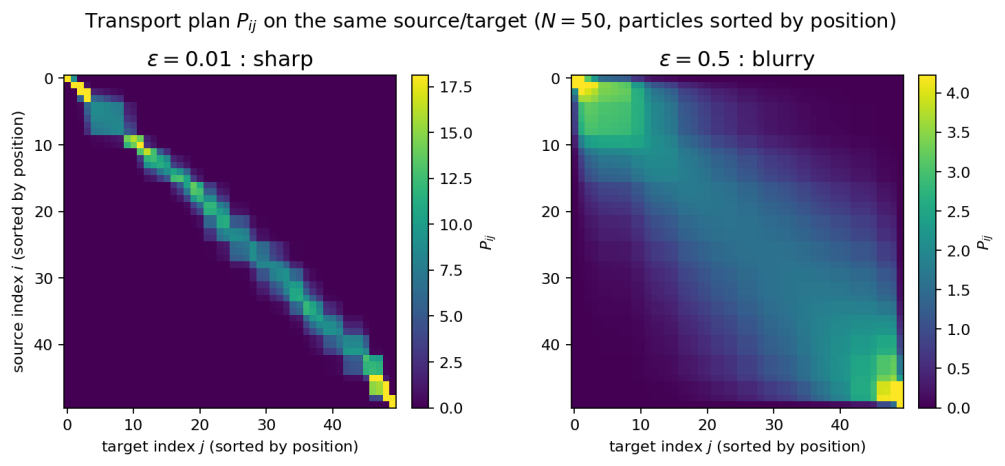
Why it converges. The entropic OT objective $\langle \mathbf{P}, \mathbf{C} \rangle - \varepsilon \mathbf{H}(\mathbf{P})$ is ε-strongly convex in $\mathbf{P}$ over the polytope $\mathcal{U}(\mathbf{a}, \mathbf{b})$. Strong convexity ensures a unique solution.

Marginal violation $\max_i |u_i^{(\ell)} K_{ij} v_j^{(\ell)} - a_i|$ vs. iteration ℓ, $N = 200$:



Sharp vs blurry transport: P_{ij} at small and large ε

Same $N = 50$ source/target on the line, particles sorted by position. Plot the converged P_{ij}:



Numerical instability: log-stabilised Sinkhorn

The problem. $K_{ij} = \exp(-C_{ij}/\varepsilon)$ underflows to 0 when C_{ij}/ε is large. Then u_i, v_j explode in the divisions. Vanilla Sinkhorn breaks for spread-out clouds or small ε.

Fix. Move to dual potentials $\alpha_i = \varepsilon \log u_i$, $\beta_j = \varepsilon \log v_j$. Then

$$P_{ij} \;=\; \exp\!\left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon}\right),$$

and the marginal-matching iteration becomes log-sum-exp (softmin):

$$\alpha_i^{\text{new}} \;=\; -\varepsilon \log \sum_j \exp\!\left(\frac{\log b_j + \beta_j - C_{ij}}{\varepsilon}\right), \quad \beta_j^{\text{new}} \;=\; -\varepsilon \log \sum_i \exp\!\left(\frac{\log a_i + \alpha_i - C_{ij}}{\varepsilon}\right).$$

Subtract row/column max before exponentiating $\Rightarrow$ numerically stable.

Damping for stability: $\alpha_i \leftarrow \frac{1}{2}(\alpha_i + \alpha_i^{\text{new}})$, same for β_j. Trades one iteration's slowdown for monotone convergence on hard instances.

How we compute the reference gradient (FD)

Central finite differences at multiple radii. For parameter value θ_0 and base step $h = 10^{-4}$:

$$s_k \;=\; \frac{L(\theta_0 + kh) - L(\theta_0 - kh)}{2kh}, \qquad k = 1, 2, 3, 4, 5$$

$$\widehat{g}_{\mathrm{FD}}(\theta_0) \;=\; \mathrm{median}\{s_1, \ldots, s_5\}$$

Median, not mean — robust to one slope being polluted by a discontinuity / cliff. We also keep the five raw s_k: if they disagree, the function isn't smooth at θ_0 and the FD gradient itself is suspect.

Autodiff side. Reverse-mode autodiff of $L(\theta)$ at the same θ_0.

Both sides use the same fixed PF seed so randomness is deterministic. FD and autodiff measure the gradient of the *same* numerical function, not two stochastic estimates.

Filter-level bias compounds

What is tested. Linear-Gaussian SSM (F=0.9, H=1, true σ_{obs}=1). LEDH-PF with OT resampling (ε=0.5), N=200, n_λ=15, always-resample (no ESS gating), fixed PF seed.

Quantity: $L(\theta) = \log p(z_{1:T} \mid \sigma_{\text{obs}}{=}\theta)$, perturbed at $\theta_0 = 1.0$.

Same code path, same forward, same FD. The only switch is which OT backward is plugged in.

T	Backward	FD median $\widehat{g}_{\text{FD}}$	Autodiff $\widehat{g}_{\text{AD}}$	Verdict
1	approx	0.008925	0.008924	rel.err. 9.3×10^{-5}
1	implicit	0.008925	0.008924	rel.err. 9.3×10^{-5}
5	approx	-1.2902	$+\mathbf{0.3690}$	**wrong sign**, rel.err. 1.286
5	implicit	-1.2902	-1.2890	rel.err. 9.4×10^{-4}
20	implicit	7.9400	7.9440	rel.err. 5.1×10^{-4}

Sources: `test_gradient_vs_numerical_lg_ot_approx.json`, `test_gradient_vs_numerical_lg.json`.

The five FD slopes at $T{=}5$ — why this is damning

Same case, $T{=}5$, $\theta_0 = 1.0$. The five central-difference slopes at radii $h, 2h, 3h, 4h, 5h$:

k	$s_k = \big(L(\theta_0 + kh) - L(\theta_0 - kh)\big)/(2kh)$
1	-1.290228679264871
2	-1.290228676813498
3	-1.290228672763405
4	-1.290228667085724
5	-1.290228659785342
median	$\widehat{g}_{\mathrm{FD}} = -1.2902286727634047$

Five slopes agreeing to seven decimal places $\Rightarrow$ $L(\theta)$ is smooth at θ_0, the true gradient is unambiguously ≈ -1.2902.

Autodiff returns:

- Approximate backward: $\widehat{g}_{\mathrm{AD}} = +0.3690$ ($\Rightarrow$ **sign and magnitude both wrong**)
- Implicit backward: $\widehat{g}_{\mathrm{AD}} = -1.2890$ ($\Rightarrow$ matches FD to 9.4×10^{-4})

Implicit backward: don't assume exact convergence

At convergence the Sinkhorn iterates (u_i, v_j) satisfy the marginal-matching system

$$F(u, v;\, a, C) \;=\; \begin{pmatrix} u_i K_{ij} v_j - a_i \\ v_j K_{ij} u_i - b_j \end{pmatrix} \;=\; 0.$$

In practice we early-stop, so $F = r \neq 0$.

Approximate (extrapolation) backward. Pretends $r = 0$. Treats (u^*, v^*) as an exact fixed point and reads gradients off one extra Sinkhorn step. Drops the residual.

Implicit backward. Linearises F at the actual returned (u^*, v^*), residual and all. Implicit function theorem gives

$$\frac{\partial (u^*, v^*)}{\partial (a, C)} \;=\; -\left(\frac{\partial F}{\partial (u, v)}\right)^{-1} \frac{\partial F}{\partial (a, C)},$$

evaluated at the returned point. One $(2N-1)$ Cholesky-grade solve.

Validation: implicit backward across models

All rows: AD vs FD ratio of $d \log p(z_{1:T} \mid \theta)/d\theta$ with the implicit Sinkhorn backward, 7 cases per row ($T \in \{1, 5, 20\}$ at the nominal parameter, plus 4 parameter values at $T = 20$).

Filter–model	Cases	Parameter sweep	Median rel. err.	Max rel. err.	Pass
LG, BPF+OT	7	$\sigma_{\text{obs}} \in \{0.5, 1.0, 1.5, 2.0\}$	1.2×10^{-4}	4.1×10^{-4}	yes
LG, LEDH+OT	7	$\sigma_{\text{obs}} \in \{0.5, 1.0, 1.5, 2.0\}$	5.1×10^{-4}	3.2×10^{-3}	yes
RB, LEDH+OT	7	$\sigma_{\text{range}} \in \{0.05, 0.1, 0.2, 0.5\}$	2.0×10^{-4}	1.4×10^{-3}	yes
SV1D, LEDH+OT	7	$\alpha \in \{0.5, 0.7, 0.91, 0.95\}$	6.5×10^{-4}	2.0×10^{-3}	yes

All rows pass with median rel. err. $< 10^{-3}$ and max $< 5 \times 10^{-3}$. The implicit backward delivers FD-accurate gradients across BPF and LEDH on linear Gaussian, range-bearing, and SV1D.

The cost wall

Component	Cost vs. multinomial
OT forward (Sinkhorn)	60–100×
Implicit backward (per resample)	one $2N-1$ Cholesky solve
Per HMC proposal	$L \cdot T \cdot$ (above)
Per chain	$n_{\text{samples}} \cdot$ (above)

Concretely: L=10, n=500, T=50 $\Rightarrow$ 250,000 resamples *per chain*.

Practical now for $T \leq 100$, $d \leq 2$. Impractical as T, N, d grow.

Why a neural operator?

Amortisation. A single HMC chain solves hundreds of thousands of *nearby* OT problems. Sinkhorn restarts each from scratch.

Train a map $\mathcal{T}_W(x;\, c)$ once, where c summarises the cloud. At inference:

- One forward pass per resample, no inner iteration.
- No $(2N-1) \times (2N-1)$ solve in the backward.
- $O(L \cdot N^2) \to O(N \cdot \text{forward\_cost})$.

The case is purely about repeated cost. Sinkhorn correctness is not at issue.

Monge–Ampère equation, intuitively

Mass conservation under change of variables. If a map $T : \mathbb{R}^d \to \mathbb{R}^d$ pushes density p onto density q, then

$$\boxed{p(x) \;=\; q\big(T(x)\big)\,\big|\det J_T(x)\big|}$$

Reading: at every x, the source mass $p(x)\,dx$ must equal the target mass $q(T(x))\,dy$ in the image; the Jacobian determinant is the local volume-change factor.

Brenier's theorem. For squared-Euclidean cost, the OT map has the form $T = \nabla\varphi$ with φ convex. Substituting:

$$p(x) \;=\; q\big(\nabla\varphi(x)\big)\,\det \nabla^2\varphi(x).$$

This is the *Monge–Ampère equation* — a fully nonlinear PDE for the potential φ.

Our route. We do not solve the PDE for φ. We parameterise the *map* $T(x; c)$ directly with a neural network and enforce the boxed identity above as a training residual. This skips the second-order Hessian of φ and gives both T and J_T from one forward pass.

Using Monge–Ampère for resampling: build source and target

Resampling input: a weighted particle cloud $\{(x_i, w_i)\}_{i=1}^N$ approximating some distribution. We want to redistribute mass so the cloud becomes equally weighted.

Why we need smooth densities. The change-of-variables identity $p(x) = q(T(x))\,|\det J_T|$ requires absolutely continuous p, q. Empirical clouds are atomic — they don't fit. **Smooth them with a Gaussian KDE** (bandwidth h, $K_h(u) = (2\pi h^2)^{-d/2} \exp(-\|u\|^2/(2h^2))$).

Source p_h — the cloud as if uniformly weighted.

$$p_h(x) \;=\; \frac{1}{N} \sum_{i=1}^{N} K_h(x - x_i)$$

Target q_h — the cloud with the importance weights.

$$q_h(x) \;=\; \sum_{i=1}^{N} w_i \, K_h(x - x_i)$$

Same particle locations, but mass concentrated where w_i is large.

Inputs and outputs of the operator

Inputs.

- **Query point** $x \in \mathbb{R}^d$ (or a batch (B, d) at training). The point at which we evaluate the map.
- **Context** $c \in \mathbb{R}^C$ summarising the weighted cloud $\{(x_i, w_i)\}_{i=1}^N$. Built by a permutation-invariant set encoder: per-particle tokens $t_i = [x_i, \log w_i, w_i]$, learned map, weighted pool $\sum_i w_i h_i$; joined with weighted mean, covariance, $\log N$, $\log \mathrm{ESS}$, $\log h$, then projected to c.

Outputs.

- **Transported point** $T(x; c) \in \mathbb{R}^d$.
- **Local Jacobian** $J_T(x; c) \in \mathbb{R}^{d \times d}$ — analytic, from the same forward pass (not autodiffed).

Both are produced together; the loss needs $|\det J_T|$ at every collocation point, so getting it from the same forward pass (instead of nesting an autodiff call) is the architectural priority.

Combining with resampling: the inference path

At every resample step in the filter:

1. Take the weighted cloud $\{(x_i, w_i)\}_{i=1}^N$.
2. Encode it once: $c \leftarrow \mathrm{SetEncoder}(\{(x_i, w_i)\})$.
3. Apply the trained map to every particle in parallel:

$$\tilde{x}_i \;=\; T(x_i;\, c), \qquad \tilde{w}_i \;=\; 1/N.$$

   Return $J_T(x_i; c)$ as well, for covariance transport in the LEDH flow.

Differentiability: $T(x; c)$ and $J_T(x; c)$ are smooth functions of their inputs; gradients flow through the cloud $\{(x_i, w_i)\}$ to θ exactly as before.

Training loss — self-supervised Monge–Ampère residual

No Sinkhorn teacher: the loss is the residual of the Monge–Ampère equation itself, evaluated at collocation points.

$$r(x;c) \;=\; \log p_h(x) \;-\; \log q_h\big(T(x;c)\big) \;-\; \log\big|\det J_T(x;c)\big|$$
$$\mathcal{L}_{\mathrm{MA}}(c) \;=\; \mathbb{E}_{x\sim p_h}\big[r(x;c)^2\big]$$

Plus two regularisers:

- **Local linearity:** $\|T(x+\delta;c) - T(x;c) - J_T(x;c)\,\delta\|^2$ on the KDE scale. Low-pass with cutoff $1/h$.
- **Eigenvalue barrier:** penalises $\lambda_{\min}(J_T)$ below threshold (keeps J_T well-conditioned, $\det > 0$).

Sinkhorn role. No Sinkhorn at training; Sinkhorn results will be used for validation.

How do we know training has converged?

Status today. Single-cloud overfit reaches $\mathcal{L}_{\mathrm{MA}} \approx 9 \times 10^{-4}$, mean transport error 0.035. *The operator can fit one cloud.* Generalisation is not yet validated end-to-end.

Three gating tests — designed, not yet run.

1. **Held-out MA residual.** Train on a Gaussian-cloud family; evaluate $\mathcal{L}_{\mathrm{MA}}$ on cloud configurations the operator never saw.
2. **Resample quality vs. Sinkhorn.** On held-out clouds, compare the operator's resampled cloud to the Sinkhorn cloud: $\mathrm{KL}(T_{\#}p_h \,\|\, q_h)$ or Sinkhorn divergence.
3. **Filter-level recovery.** Drop the operator into LEDH+OT on linear Gaussian; check posterior recovery against the Sinkhorn-baseline HMC. The end-to-end test.

Until tests 1–3 pass, "low training loss" is not "trained operator". The dataset pipeline (100 Gaussian-grid clouds, 70/15/15 split) is the gating work.

Determinant in higher dimension — forward and backward

The concern: $\det J_T$ is $O(d^3)$ for a generic $d \times d$ matrix, and the derivative makes it worse. How do we keep this from blowing up?

Architectural fix — structured Jacobian. We never let J_T be a generic dense matrix. We force the structure

$$J_T(x; c) \;=\; \lambda(c)\, I \;+\; \sum_m W_m^\top D_m(x; c)\, W_m,$$

with $D_m \succeq 0$ diagonal. By construction J_T is symmetric PSD, so $\det J_T > 0$ and $\log|\det J_T| = \log\det J_T$ (no sign tracking).

Forward det **via Cholesky.** $O(d^3)$ per query point in the worst case — but trivial at $d \leq 2$ (every model in this report).

Backward through $\log\det$**.** The identity $\partial \log\det A/\partial A = A^{-\top}$. *One linear solve, not a Hessian.* Cholesky factor from the forward is reused $\Rightarrow$ one triangular solve, same $O(d^3)$. The derivative is no harder than the forward.

Escape hatch for high d**.** Take each W_m to be rectangular $r \times d$ with $r \ll d$. Then

Should θ be an input?

The case for adding θ. HMC moves θ every proposal; the cloud the filter produces depends on θ. So in principle conditioning the operator on θ gives it more information.

Why we don't, by default.

- Different parametric families have different θ. Conditioning ties the operator to one family — breaks the goal of a model-agnostic operator.
- Even within one family, θ alone does *not* pin down the cloud: at fixed θ the filter produces a *different* cloud at every $t = 1, \ldots, T$. Conditioning on θ is not sufficient information.
- The cloud $\{(x_i, w_i)\}$ already carries the effect of θ. As long as the cloud is a sufficient summary, the operator does not need θ.

Precondition for adding θ. It would help only if two trajectories $\gamma_{\theta_1}, \gamma_{\theta_2}$ pass through clouds that look the same in (x, w)-space but require *different* transport maps. Empirical check: scan held-out (cloud, target) pairs; if cloud-only residual correlates with θ at fixed c, the cloud is not sufficient and θ should be added. Until that check fails, cloud-only is the parsimonious default.

JIT-compiling the operator inside HMC

Why it's hard.

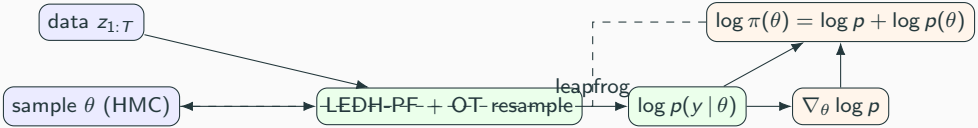
- The operator is a learned module with weights and (likely) Keras layers. Any single non-XLA kernel inside breaks the trace.
- Operator weights are *frozen* at HMC time. If the trace records weight reads as gradient sources, autodiff traces through them needlessly.
- The full HMC graph stacks: filter forward $\rightarrow$ operator at every resample $\rightarrow$ analytic Jacobian $\rightarrow$ autodiff back to θ. One non-XLA op anywhere kills the whole compilation.
- The training loss uses operations like `slogdet` and `eigvalsh`; the first has no XLA-CPU kernel, the second is brittle inside JIT.

What we do.

- Single inner JIT-compiled function: filter forward + operator calls + analytic Jacobian + autodiff back to θ. Operator weights stop-gradient'd on weight reads (not outputs).
- Replace `slogdet` with the same QR-based log-abs-det used for LEDH.
- Drop training-only regularisers (eigenvalue barrier, local linearity) from the inference graph

HMC + LEDH + OT pipeline

The full pipeline



All boxes inside one JIT-compiled graph; gradient computed by reverse-mode autodiff on the same trace.

HMC setup — the choices that matter

- **Sampler:** Hamiltonian Monte Carlo with dual averaging on the step size; target acceptance 0.75–0.80.
- **Leapfrog:** $L = 10$ for SV1D / RB; per-axis step in RB.
- **Chains:** 4 independent, separate DA inits. (Single-chain ESS $= 4.0$ on SV1D was a DA-init artefact, not a real failure — 4 chains converge.)
- **Bijectors** for the SVSSM:
  - μ: identity
  - ϕ: tanh ($\mathbb{R} \to (-1, 1)$)
  - σ_η: softplus
- **Diagnostics:** split-$\hat{R} \leq 1.01$ (Vehtari et al. 2021), bulk + tail ESS ≥ 100 per parameter per chain, energy diagnostics.

Priors for the SVSSM

- $\mu \sim \mathcal{N}(0, 5^2)$ — weakly informative; log-volatility mean is negative for typical assets but should not be hard-bounded.
- $\phi \sim \mathrm{Beta}(20, 1.5)$ mapped to $(-1, 1)$ — support concentrated near 0.95–0.99, matches financial empirics.
- $\sigma_\eta \sim \mathrm{LogNormal}(\log \sigma_\eta^* + 0.25,\ 0.5)$ — mode at the true value (LogNormal mode quirk: $\exp(\mu - s^2)$).

Why this matters for HMC: the prior's tail behaviour interacts with the bijector — weakly identified directions can stretch the leapfrog trajectory past the typical set.

Initial condition h_0

Stationary ($|\phi| < 1$):

$$h_0 \sim \mathcal{N}\left(\mu,\ \frac{\sigma_\eta^2}{1-\phi^2}\right)$$

Non-stationary ($|\phi| \geq 1$): diffuse $h_0 \sim \mathcal{N}(0, \kappa)$ with large κ.

The trap. The stationary form makes $\Sigma_0 = \sigma_\eta^2/(1-\phi^2)$ a function of (ϕ, σ_η). Differentiating through the Cholesky $\rightarrow$ exp chain explodes near $\phi \rightarrow 1$.

This codebase: detach h_0 from transition parameters via numpy init. Slight prior bias, removes the gradient pathology that froze HMC at high persistence.

Benchmark step 1: linear-Gaussian HMC, Kalman / EKF / UKF

Same linear-Gaussian observation sequence ($\sigma^*_{\text{obs}} = 1.0$). Four chains each, dual averaging, target acceptance 0.65.

Filter	Posterior mean	Sd	90% CI	Max $\hat{R}$	Bulk ESS	Tail ESS
Kalman	1.127	0.177	[0.85, 1.44]	1.008	278	325
EKF	1.142	0.176	[0.87, 1.46]	1.003	419	518
UKF	1.138	0.173	[0.87, 1.45]	1.002	464	494

All three converge: $\max \hat{R} \leq 1.008$ on rank-normalised samples, bulk + tail ESS in the hundreds. Posterior means agree to within 0.015 on the same data; the ~ 0.13 offset above the truth is dataset-specific. This is the convergence target for the differentiable-filter HMC runs that follow.

Diagnostics: ESS and rank-normalised $\hat{R}$

Effective sample size (ESS). For a chain of N samples of a parameter θ with lag-k autocorrelation $\hat{\rho}_k$,

$$\mathrm{ESS} \;=\; \frac{N}{1 + 2\sum_{k \geq 1} \hat{\rho}_k}.$$

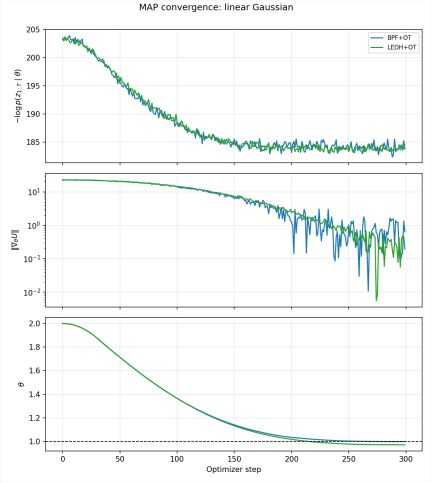
Counts effectively-independent samples. **Bulk ESS** is computed on the rank-normalised pooled samples; **tail ESS** on the indicators $\mathbb{1}\{\theta < q_{0.05}\}$ and $\mathbb{1}\{\theta > q_{0.95}\}$ (catches tail-mixing failures bulk ESS misses). Threshold: bulk + tail ESS ≥ 100 per parameter per chain.

Rank-normalised $\hat{R}$ (Vehtari et al. 2021). Pool all MN samples, replace each by its rank, transform to standard normal: $z_t^{(m)} = \Phi^{-1}\big(\mathrm{rank}(\theta_t^{(m)})/(MN+1)\big)$. Run the standard between-/within-chain variance ratio on z:

$$\hat{R} \;=\; \sqrt{\frac{\widehat{\mathrm{Var}}(z)}{W(z)}}, \qquad \widehat{\mathrm{Var}}(z) = \frac{N-1}{N} W(z) + \frac{1}{N} B(z).$$

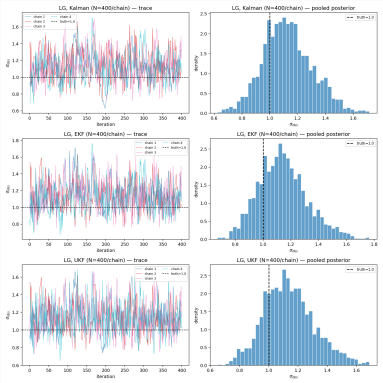
Rank normalisation makes the test sensitive to *any* cross-chain distributional difference, not just mean/variance.

Benchmark step 2: MAP sanity — BPF+OT and LEDH+OT on LG



Benchmark step 3: HMC on linear Gaussian

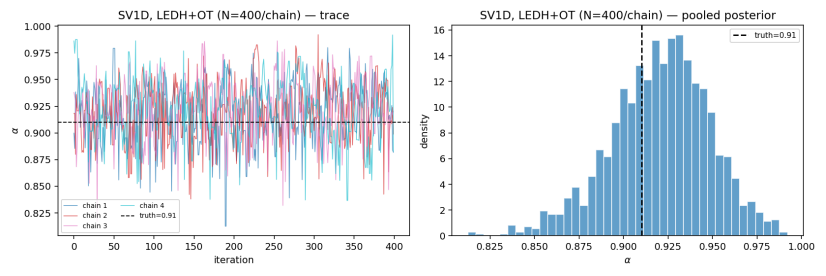
Same LG setup. Run HMC on σ_{obs} with both BPF+OT and LEDH+OT, four chains each, separate dual-averaging inits. Compare to the Kalman family baseline.



Posteriors from BPF+OT and LEDH+OT match the Kalman / EKF / UKF posteriors. $\hat{R} \leq 1.01$, bulk + tail ESS ≥ 100. The differentiable particle-filter pipeline reproduces the

Benchmark step 4: HMC on SV1D

Stochastic volatility 1D, log-volatility latent h_t, observation $z_t = \exp(h_t/2)\, v_t$. No closed-form posterior. Recover (μ, ϕ, σ_η) from $z_{1:T}$.



Four chains converge cleanly. Bulk + tail $\hat{R} \leq 1.01$ across all three parameters. Posterior medians at the truth, contraction relative to prior visible.

Profiling commitments

Before claiming any speedup:

- Profile a 10-leapfrog HMC window for LEDH+OT on the largest model in the benchmark set. Breakdown:
  1. Flow accumulation $T \times n_\lambda \times N$
  2. Sinkhorn forward
  3. Implicit backward solve
  4. Parameter-Jacobian path
  5. Framework overhead
- Cross-check: 1D / 2D models are op-launch bound ($\sim 10\,\mu$s per op); only 5D+ shows true algorithmic scaling.
- Memory: confirm post-fix steady state. Earlier leak was 13–18 GB per call from particle initialisation inside the marginal-likelihood routine.

Measured XLA win: $\sim 1.3\times$ forward at $T{=}20$. **Do not promise more before measuring.**

Identifiability when $z_t = Ax_t + \ldots$

Scalar a: clean. a enters linearly in the mean, $\partial \log p / \partial a = (z_t - a x_t) x_t / R_t$. EKF / BPF / LEDH all work.

Matrix A ($d \geq 2$): (A, x_t) identifiable only up to (AQ^{-1}, Qx_t) for invertible Q. Fix by canonical form:

- Lower-triangular A with positive diagonal — easiest with a fill-scale-triL bijector.
- Stiefel manifold (orthonormal columns) — needs manifold sampler.
- First entry of each column fixed to 1 — factor-model loading normalisation.

Rank conditions: $\dim(y) > \dim(h)$ identifiable up to column rotation; $\dim(y) < \dim(h)$ leaves part of h unobservable.

Bonus: Hamiltonian Neural Networks

Idea (arxiv 2208.06120): train an HNN on $(\theta, \log p(y \mid \theta), \nabla_\theta \log p)$ pairs from real HMC chains; use HNN-supplied ∇H in leapfrog instead of running the filter every step.

Why interesting. Same amortisation argument as the OT operator, but at the *posterior* level.

Why not now.

- HNN needs accurate gradients — noisy filter gradients in, garbage HNN out.
- Distributional shift: HNN generalises only inside training support; the posterior region is exactly what we don't know.
- Doesn't eliminate the filter — still need MH correction with real H.

Future direction, not Phase-2.

Thank you